

AI Operations Assignment 2

Krish Yadav
DA24B043

Q1. Docker Image Optimization

Part 1: Single-Stage Build

I used a single-stage `python:3.12-slim` Dockerfile, installed the required packages and copied the application files. The image size was **723 MB**. The container successfully served `/healthz` and `/predict`.

Part 2: Multi-Stage Build

I then used a builder stage to install the Python dependencies and copied the installed environment into the final `python:3.12-slim` image. The measured image sizes were

Single-stage = 723 MB, Multi-stage = 710 MB

Hence, the percentage reduction was

$$\frac{723 - 710}{723} \times 100 = \mathbf{1.80\%}$$

The multi-stage image also passed the same API tests.

Part 3: Explanation

In the multi-stage build, dependencies are installed under `/install` in the builder stage, but the builder stage itself, including its intermediate filesystem and layers, is not included in the final image. Only the dependencies and application files are copied to the final stage, reducing the image from 723 MB to 710 MB.

Q2. Docker Compose and Redis Caching

Part 1: Redis and Compose

I added Redis caching to the API. The API first checks Redis using the exact input text as the key. On a miss, it computes the prediction, stores the label with a TTL of 300 seconds, and returns it; on a hit, the cached label is returned without recomputing.

I used two Compose services: `api`, built from the multi-stage Dockerfile, and `cache`, using `redis:7-alpine`. The API connects to Redis using `cache` as the hostname.

Part 2: Cache Timing

After clearing the Redis database, I sent the same request twice:

```
{"text": "WIN a FREE laptop now!"}
```

Both requests returned `{"label": "spam"}`. The measured times were

First (MISS) = 0.012268s, Second (HIT) = 0.001506s

Thus, the second request was

$$\frac{0.012268}{0.001506} \approx \mathbf{8.15\times}$$

faster than the first request.

Part 3: Compose vs Kubernetes

Docker Compose runs related containers together and provides their networking on a single host. Kubernetes provides cluster-scale orchestration across multiple nodes, including scheduling and self-healing.

Q3. Kubernetes Indexed Job

Part 1: Shards and Parallelism

I generated 8 deterministic CSV shards with 20 rows each and 3 deliberately invalid rows per shard. The validator uses the shard index to select exactly one shard and reports its invalid-row count and node name.

The assignment assumes 2 nodes with 2 CPUs each, giving 4 CPUs total. I chose **parallelism = 4**, with a request and limit of **1 CPU per pod**. The Job uses **completions: 8** and Indexed completion mode. The shard index and node name are passed using the Downward API.

`kubectl get pods -o wide` showed 4 pods running concurrently across `minikube` and `minikube-m02`, confirming the chosen parallelism.

Part 2: Results

The Kubernetes API was used to read the logs of all completed pods. The invalid-row counts were:

Shard	0	1	2	3	4	5	6	7
Invalid rows	3	3	3	3	3	3	3	3

Therefore, the total was **24 invalid rows**.

I used pod logs instead of a shared volume because the Minikube **standard** StorageClass uses the `k8s.io/minikube-hostpath` provisioner, which provides node-local storage. Logs allow each pod to report its result directly without shared storage.

Part 3: If 6 CPUs Were Available

With 3 nodes and 2 CPUs each, there would be 6 CPUs available. I would increase parallelism to **6**, keeping the 1 CPU request and limit, so up to 6 of the 8 pods could run concurrently.

Q4. Kubernetes Deployment

Part 1: Deployment and Service

I deployed the spam-detection API using a Kubernetes Deployment with **2 replicas**. Each pod has a CPU request and limit of **1 CPU** and a readiness probe checking `/healthz`. A NodePort Service was created to expose the API.

Part 2: Self-Healing

I manually deleted one running API pod. Kubernetes recreated a replacement, which became **1/1 Running**. The Deployment manages a ReplicaSet, which maintains the desired number of replicas and creates a replacement when a pod is deleted.

Part 3: Rolling Update

I added `"version": "v2"` to the `/healthz` response, built `spam-api:v2`, and updated the Deployment. The **rollout status** completed successfully, **rollout history** showed revisions 1 and 2, and both new replicas were running. The readiness probe ensured that new pods were ready during the update.

If this application had been deployed as a Job instead, Kubernetes would treat the pods as run-to-completion tasks rather than maintaining serving replicas, so there would be no Deployment-style rolling update. Updating the image would apply to newly created Job pods rather than gradually replacing the running API replicas.

Part 4: Deployment vs Job

A Deployment maintains replicas of a long-running application and provides self-healing and rolling updates, making it suitable for serving the spam API. A Job runs pods until a finite workload is completed, making it suitable for the shard validator batch processing.